

Informed Search

Instructional Objectives

- Understand what is heuristic function
- Be able to design heuristic function for a given problem
- Be able to prove that if A^* uses an admissible heuristic function, it will produce an optimum solution
- Should learn how to compare two heuristic functions

Informed Search

- We have seen that uninformed search methods that systematically explore the state space and find the goal
 - Inefficient in most cases
 - Informed search methods use problem specific knowledge, are more efficient

Heuristics

- Heuristic: “ rule of thumb”
- “ Heuristics are criteria, methods or principles for deciding which among several alternative courses of action promise to achieve the most effective in order to achieve some goal” – Judea Pearl
- Can use heuristics to identify the most promising search path

Example of Heuristic Function

- A heuristic function at a node n is an estimate of the optimum cost from the current node to a goal. Denoted by $h(n)$
- $h(n)$ = estimated cost of the cheapest path from node n to a goal node
- Example:
- Want path from Kolkata to Guwahati
- Heuristic for Guwahati may be straight line distance between Kolkata and Guwahati
- $h(\text{kolkata}) = \text{euclideanDistance}(\text{Kolkata}, \text{Guwahati})$

Heuristic Example

- 8-puzzle: No. of tiles out of place
- Manhattan Distance

Informed Search Strategies

- Best-first search
 - Greedy search
 - A* search
- Heuristics
- Hill-climbing
- Simulated Annealing

Best-first search

Idea: use an *evaluation function* for each node to estimate its 'desirability'

- → Expand most desirable node in queue
- Implementation:
 - QueueingFn: insert successors in decreasing order of desirability

Best-first Search

- Generalization of breadth first search
 - Priority queue of nodes to be explored
 - Cost function $f(n)$ applied to each node
- $f(n)$ is the cost function which denotes the priority of the queue
- Nodes are put in fringe and sorted according to the value of $f(n)$

Best-first Search Algorithm

Let fringe be a priority queue containing the initial state

Loop

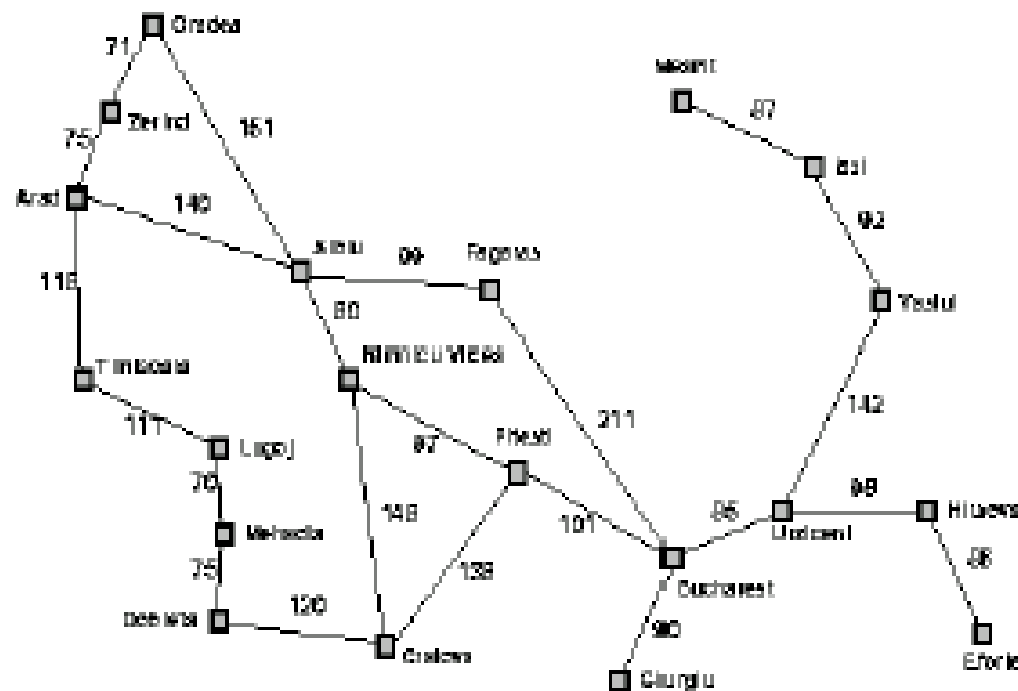
```
if fringe is empty return failure
Node → remove first (fringe)
if Node is a goal
    then return the path from initial state to Node
else generate all successors of node and
    put the newly generated nodes into fringe
    according to their f value
```

End

Implementation of Search Algorithms

```
Function General-Search(problem, Queueing-fn) return  
    solution/failure  
nodes ← MAKE-QUEUE(Initial-State(problem))  
loop do  
    if nodes is empty return failure  
    node ← REMOVE-FRONT(nodes)  
if (GOAL-TEST(problem) applied to STATE(node)  
    succeeds  
then return node  
nodes ← QUEUEING-FN(nodes, EXPAND(node,  
    OPERATORS[  
    problem]))
```

Example Romania



Straight-line distance to Bucharest

Arad	366
Bucharest	0
Oradea	160
Drobeta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	319
Urziceni	80
Vaslui	199
Zerind	374

Greedy Search Algorithm

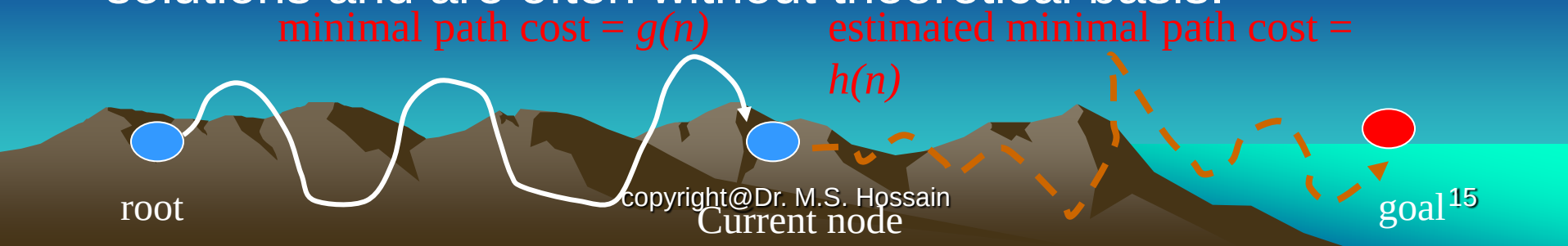
- Different variations of best-first search according to function f
- Idea: expand node with the smallest estimated cost to reach the goal
- Use heuristic function $f(n) = h(n)$
 - Not optimal
 - incomplete

Greedy Search

- Evaluation function $h(n)$, also called *heuristic* is an estimate of cost from n to *goal*
- E.g. $h_{\text{SLD}}(n)$ = straight-line distance from n to Bucharest
- Greedy search expands the node that *appears* closest to goal

Informed Methods Add Domain-Specific Information

- Add domain-specific information to select what is the best path to continue searching along
- Define a heuristic function, $h(n)$, that estimates the "goodness" of a node n with respect to reaching a goal.
- Specifically, $h(n)$ = estimated cost (or distance) of minimal cost path from n to a goal state.
- **$h(n)$ is about cost of the future search, $g(n)$ past search**
- $h(n)$ is an **estimate** (rule of thumb), based on domain-specific information that is **computable** from the current state description. Heuristics do not guarantee feasible solutions and are often without theoretical basis.



Heuristics

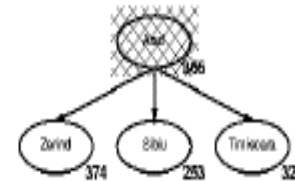
- Examples:
 - 8-puzzle: Number of tiles out of place (i.e., not in their goal positions)
- In general:
 - $h(n) \geq 0$ for all nodes n
 - $h(n) = 0$ implies that n is a goal node
 - $h(n) = \text{infinity}$ implies that n is a deadend from which a goal cannot be reached

Greedy Search Example



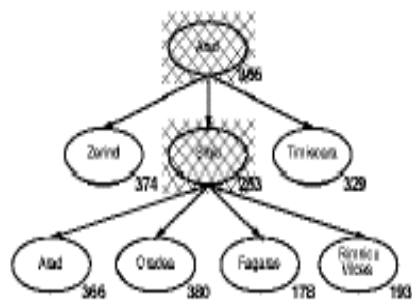
Queue: Arad (366)

Greedy Search Example



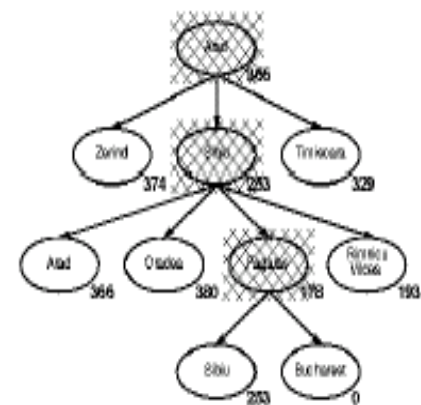
Queue: Sibiu (253), Timisoara (329), Zerind (374)

Greedy Search Example



Queue: Fagaras (178), Rimnicu (193), Timisoara (329), Arad (366),
Zerind (374), Oradea(380)

Greedy Search Example



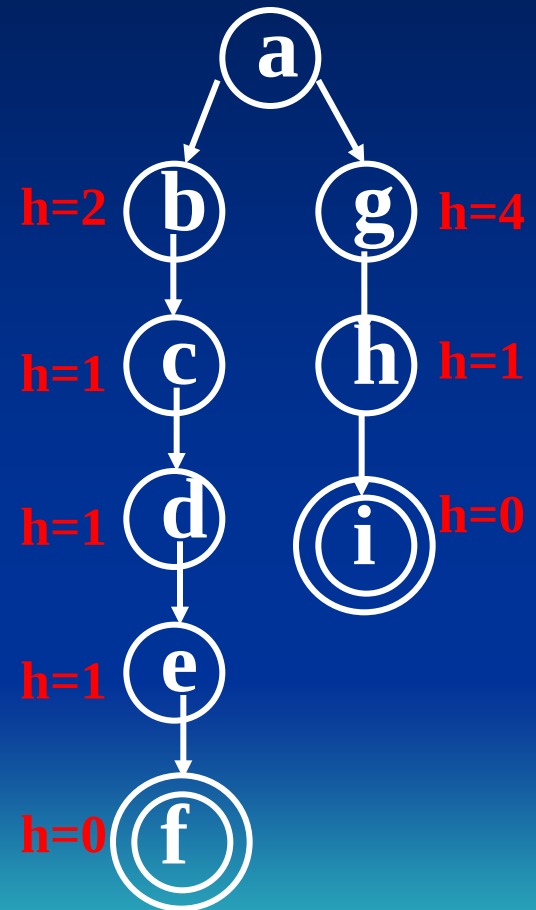
Queue: Bucharest (0), Rimnicu (193), Sibiu (253),
Timisoara (329), Arad (366), Zerind (374), Oradea(380)

Best First Search

- Order nodes on the OPEN list by increasing value of an evaluation function, $f(n)$, that incorporates domain-specific information in some way.
- Example of $f(n)$:
 - $f(n) = g(n)$ (uniform-cost)
 - $f(n) = h(n)$ (greedy algorithm)
 - $f(n) = g(n) + h(n)$ (algorithm A)
- This is a generic way of referring to the class of informed methods.

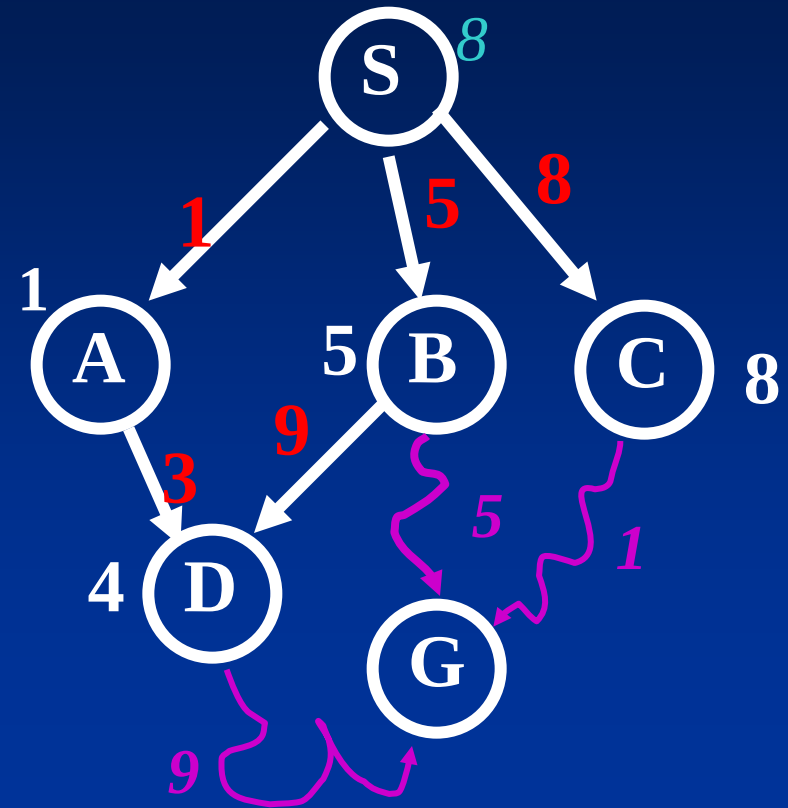
Greedy Search

- Evaluation function $f(n) = h(n)$, sorting open nodes by increasing values of f .
- Selects node to expand believed to be closest (hence it's "greedy") to a goal node (i.e., smallest $f = h$ value)
- Not admissible, as in the example. Assuming all arc costs are 1, then Greedy search will find goal f , which has a solution cost of 5, while the optimal solution is the path to goal i with cost 3.
- Not complete (if no duplicate check)



Algorithm A

- Use as an evaluation function
 $f(n) = g(n) + h(n)$
- The $h(n)$ term represents a “depth-first” factor in $f(n)$
- $g(n)$ = minimal cost path from the start state to state n generated so far
- The $g(n)$ term adds a "breadth-first" component to $f(n)$.
- Ranks nodes on OPEN list by estimated cost of solution from start node through the given node to goal.
- Completeness and admissibility depends on $h(n)$



$$f(D) = 4 + 9 = 13$$

$$f(B) = 5 + 5 = 10$$

$$f(C) = 8 + 1 = 9$$

**C is chosen
next to expand**

Algorithm A

OPEN := {S}; CLOSED := {};

repeat

Select node n from OPEN with minimal $f(n)$ and place n on CLOSED;

if n is a goal node **exit** with success;

Expand(n);

For each child n' of n do

 compute $h(n')$, $g(n')=g(n)+c(n,n')$, $f(n')=g(n')+h(n')$;

if n' is not already on OPEN or CLOSED **then**

 put n' on OPEN; set backpointer from n' to n

else if n' is already on OPEN or CLOSED and **if** $g(n')$ is lower for the new version of n' **then**

 discard the old version of n' ;

 Put n' on OPEN; set backpointer from n' to n

until OPEN = {};

exit with failure

Algorithm A*

- Algorithm A with constraint that $h(n) \leq h^*(n)$
- $h^*(n)$ = true cost of the minimal cost path from n to any goal.
- $g^*(n)$ = true cost of the minimal cost path from S to n .
- $f^*(n) = h^*(n) + g^*(n)$ = true cost of the minimal cost solution path from S to any goal going through n .
- h is **admissible** when $h(n) \leq h^*(n)$ holds.
- Using an admissible heuristic guarantees that the first solution found will be an optimal one.
- A* is **complete** whenever the branching factor is finite, and every operator has a fixed positive cost (total # of nodes with $f(.) \leq f^*(\text{goal})$ is finite)
- A* is **admissible**

Properties of A* Search

- Completeness: Yes - unless there are infinitely many nodes with $f \leq f(G)$
- Time complexity: Exponential in [rel. error in $h \times$ length of solution].
- Space complexity: Same as time complexity - keeps all nodes in memory.
- Optimality: Yes.

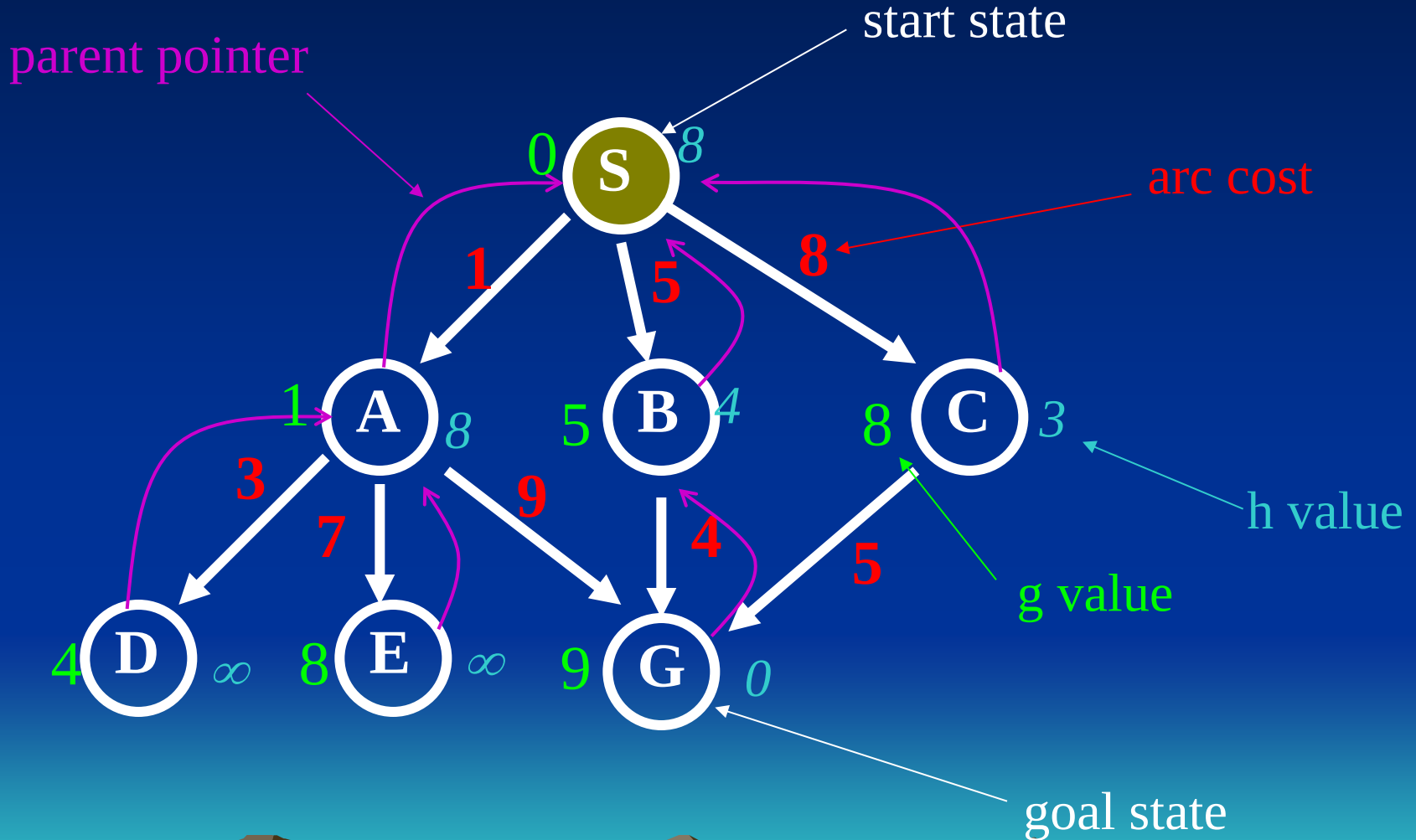
Admissibility of A*

- Admissibility provided a solution exists, the first solution found is a optimal solution
- Condition for admissibility
 - State-space graph
 - Every node has a finite number of successor
 - Every arc in the graph has a cost greater than some $\epsilon > 0$
 - Heuristic function
 - For every node n , $h(n) < h^*(n)$

Admissibility of A*

- A* is optimally efficient for a given heuristic
- If the optimal search algorithms that expand search paths from the new root node, it can be shown that no other optimal algorithm will expand fewer nodes and find a solution
- Monotonic heuristic: along any path the f-cost never decreases
- If this property does not hold we can use the following trick (m is a child of n)
- $f(m) = \max(f(n), g(m) + h(m))$

Example search space



Example

n	$g(n)$	$h(n)$	$f(n)$	$h^*(n)$
S	0	8	8	9
A	1	8	9	9
B	5	4	9	4
C	8	3	11	5
D	4	inf	inf	inf
E	8	inf	inf	inf
G	9	0	9	0

- $h^*(n)$ is the (hypothetical) perfect heuristic.
- Since $h(n) \leq h^*(n)$ for all n , h is admissible
- Optimal path = S B G with cost 9.

Greedy Algorithm

$$f(n) = h(n)$$

node	exp.	OPEN list
		{S(8)}
	S	{C(3) B(4) A(8)}
	C	{G(0) B(4) A(8)}
	G	{B(4) A(8)}

- Solution path found is S C G with cost 13.
- 3 nodes expanded.
- Fast, but not optimal.

A* Search

$$f(n) = g(n) + h(n)$$

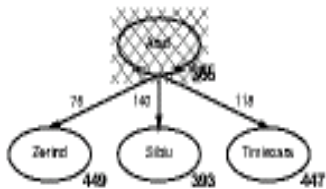
node	exp.	OPEN list
		{S(8)}
S		{A(9) B(9) C(11)}
A		{B(9) G(10) C(11) D(inf) E(inf)}
B		{G(9) G(10) C(11) D(inf) E(inf)}
G		{C(11) D(inf) E(inf)}

- Solution path found is S B G with cost 9
- 4 nodes expanded.
- Still pretty fast. And optimal, too.

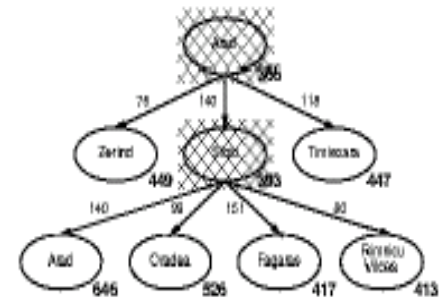
Example: A* search



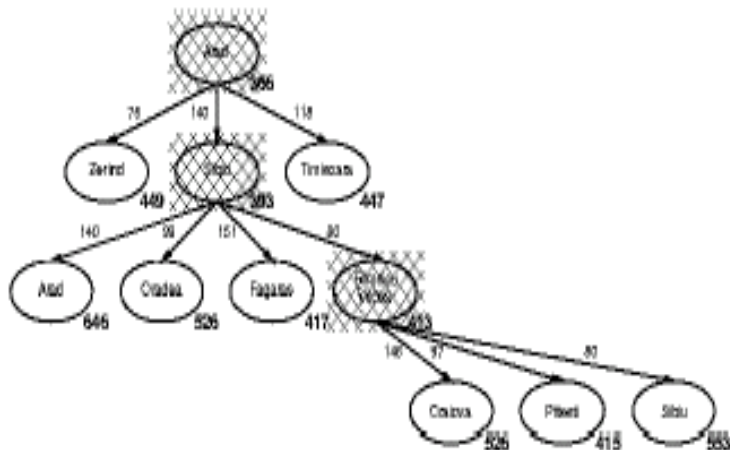
Example: A* search



Example: A* search



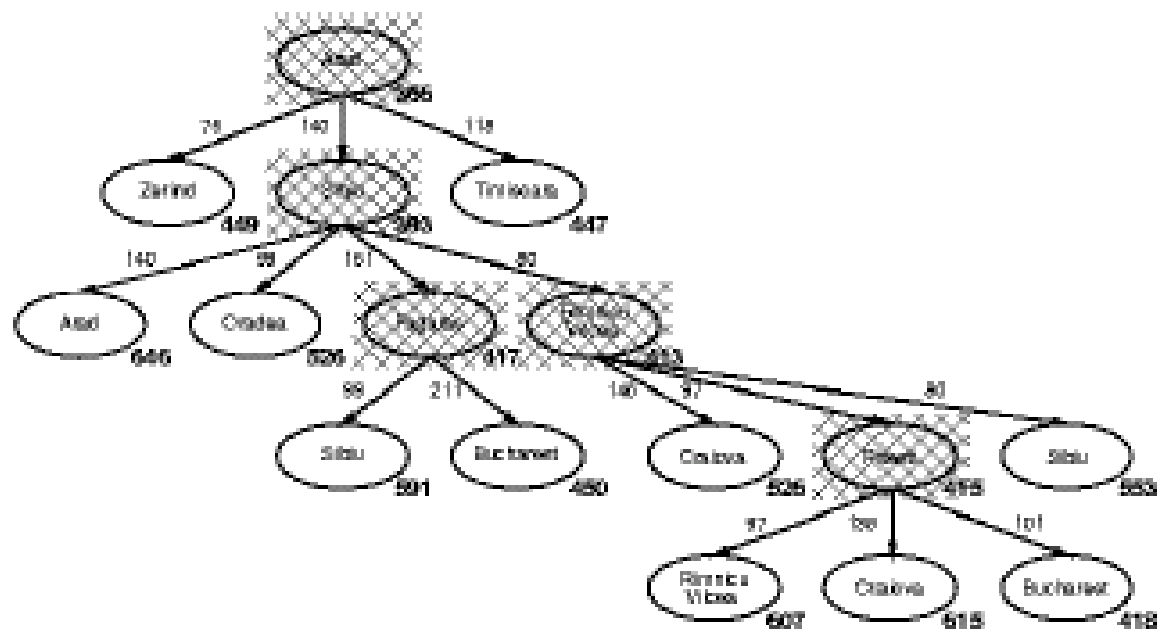
Example: A* search



Example: A* search



Example: A* search

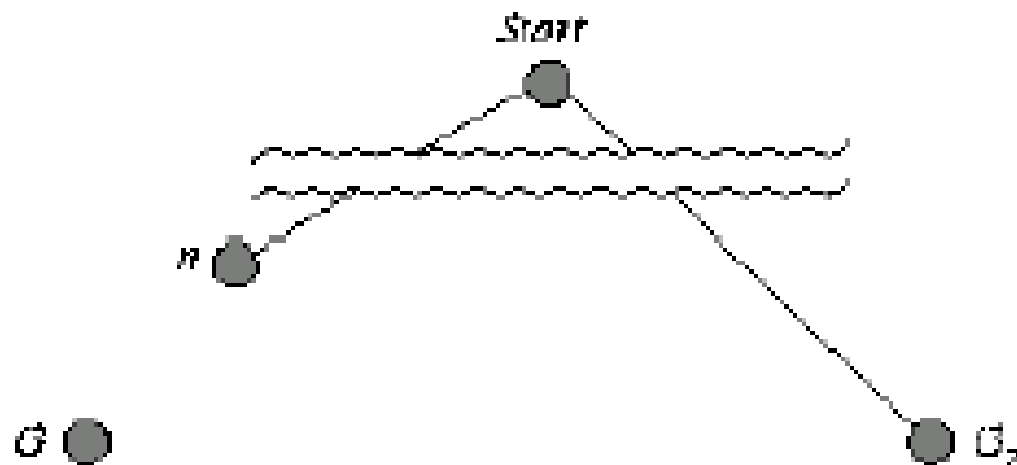


Proof of the Optimality of A*

- Let l^* be the **optimal** solution path (from S to G), let f^* be its cost
- At any time during the search, one or more node on l^* are in OPEN
- We assume that A* has selected G2, a goal state with a suboptimal solution ($g(G2) > f^*$).
- We show that this is impossible.
 - Let node n be the shallowest OPEN node on l^*
 - Because all ancestors of n along l^* are expanded, $g(n)=g^*(n)$
 - Because $h(n)$ is admissible, $h^*(n) \geq h(n)$. Then
$$f^* = g^*(n) + h^*(n) \geq g^*(n) + h(n) = g(n) + h(n) = f(n).$$
 - If we choose G2 instead of n for expansion, $f(n) \geq f(G2)$.
 - This implies $f^* \geq f(G2)$.
 - G2 is a goal state: $h(G2) = 0$, $f(G2) = g(G2)$.
 - Therefore $f^* \geq g(G2)$
 - Contradiction.

Optimality of A* search

Suppose there is a suboptimal solution G_2 being added to the queue.
Let n be a node on a path to the optimal solution G .



$$\begin{aligned} f(G_2) &= g(G_2), \text{ since } h(G_2)=0. \\ &> g(G), \text{ since } h(G_2) \text{ is suboptimal.} \\ &\geq f(n), \text{ since } h \text{ is admissible} \end{aligned}$$

Since $f(G_2) > f(n)$, A* will never select G_2 for expansion.

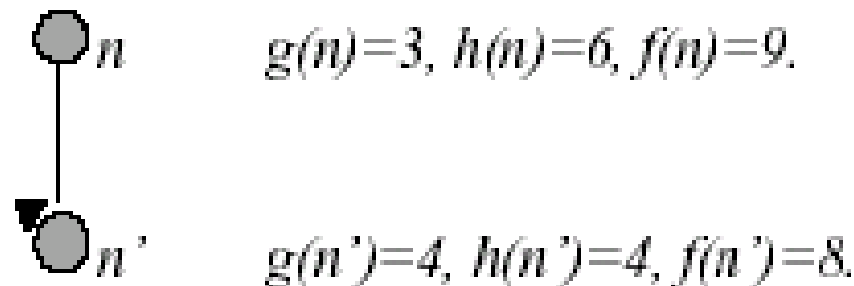
Completeness of A*

- Let G be an optimal goal state
- A* can't reach a goal state only if there are infinitely many nodes where $f(n) \leq f^*$
- Can only happen if either happens:
 - A node with infinite branching factor
 - First condition take care of this
 - A path with finite cost but infinitely many nodes
 - All arc costs are $>g$. Thus if there are infinitely many nodes on a path $g(n) > f^*$

Proof of Lemma Pathmax

For some admissible heuristic, f may decrease along a path.

E.g. suppose n' is the successor node of n .



But this throws information away.

$f(n)=9 \implies$ true cost of path through $n \geq 9$. Hence true cost through n' is also ≥ 9 .

Pathmax modification to A^* :

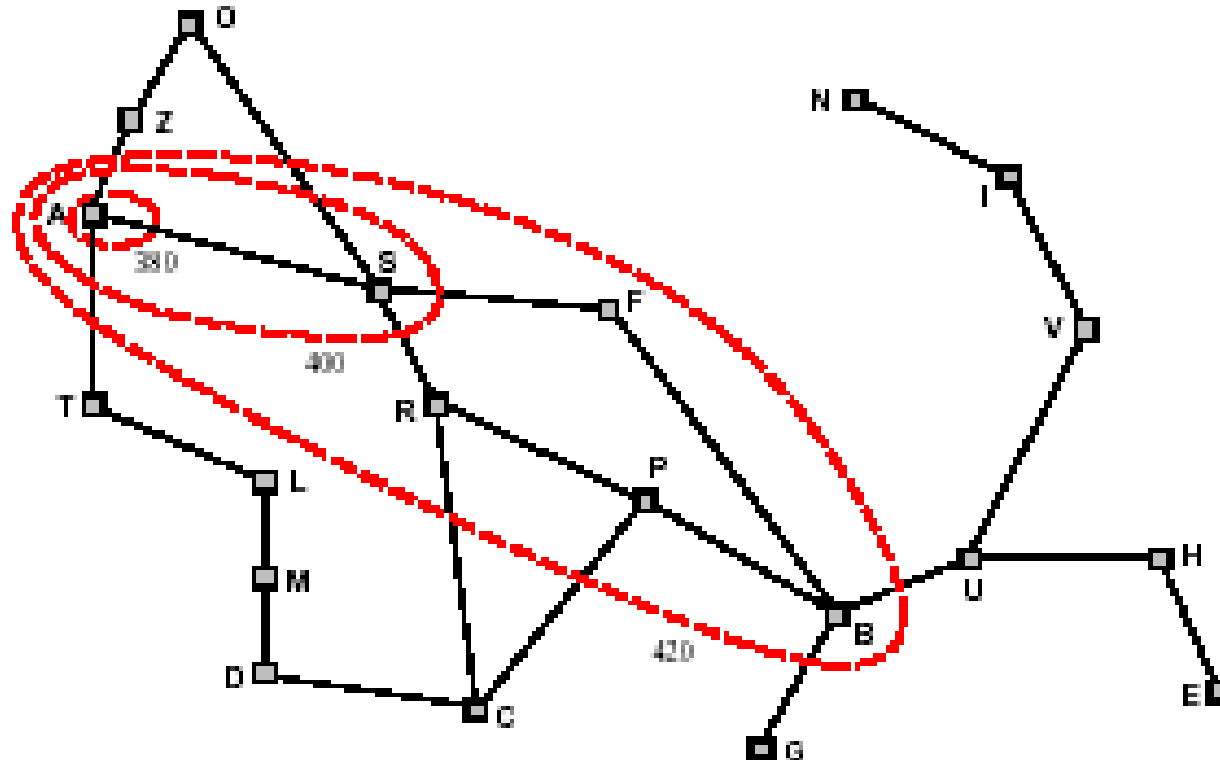
Instead of $f(n')=g(n')+h(n')$ use $f(n')=\text{MAX}(g(n')+h(n'), f(n))$.

Optimality of A* (more useful)

Lemma: A* expands nodes in order of increasing f value*

Gradually adds " f -contours" of nodes (cf. breadth-first adds layers)

Contour i has all nodes with $f = f_i$, where $f_i < f_{i+1}$



Example of a heuristic:

E.g., for the 8-puzzle:

$h_1(n)$ = number of misplaced tiles

$h_2(n)$ = total **Manhattan** distance
(i.e., no. of squares from desired location of each tile)

7	2	4
5		6
8	3	1

Start State

1	2	3
4	5	6
7	8	

Goal State

$$h_1(n) = 6$$

$$h_2(n) = 4 + 0 + 3 + 3 + 1 + 0 + 2 + 1 = 14$$

Which heuristic function, $h_1(n)$, $h_2(n)$, is **better**?

In general, a better heuristic function leads you to the solution faster. That is, it requires **fewer** node expansions.

Effective branching factor : With N node expanded and a goal node reached at depth d , the effective branching factor b satisfies:

$$N + 1 = 1 + b + b^2 + \dots + b^d$$

For example, if $N=14$ and $d=3$, effective branching factor is 2.

A good heuristic should have b close to 1.

Dominance

If $h_2(n) \geq h_1(n)$ for all n (both admissible)
then h_2 *dominates* h_1 and is better for search

Typical search costs:

$d = 14$ IDS = 3,473,941 nodes

$A^*(h_1) = 539$ nodes

$A^*(h_2) = 113$ nodes

$d = 24$ IDS $\approx$ 54,000,000,000 nodes

$A^*(h_1) = 39,135$ nodes

$A^*(h_2) = 1,641$ nodes

Iterative Deepening A* (IDA*)

- **Idea:**
 - Similar to IDDF except now at each iteration the **DF search** is not bound by the current `depth_limit` but by the current **f_limit**
 - At each iteration, all nodes with **f(n) ≤ f_limit** will be expanded (in DF fashion).
 - If no solution is found at the end of an iteration, **increase f_limit** and start the next iteration
- **f_limit:**
 - Initialization: `f_limit := h(s)`
 - Increment: at the end of each (unsuccessful) iteration, `f_limit := max{f(n) | n is a cut-off node}`
- **Goal testing:** test all cut-off nodes until a solution is found
select the least cost one if there are multiple solutions
- **Admissible** if h is admissible

IDA* Analysis

- Complete and optimal
- Space usage proportional to depth of solution
- Each iteration is DFS – no priority queue
- # nodes expanded relative to A*
 - Depends on # unique values of heuristic function

IDA* Analysis

- # nodes expanded relative to A*
 - In 8-puzzle few values close to
- #A* expands
 - In travelling salesman, each f value is unique
 - $1+2+.....n =$
- A* expands
 - If n is too big for main memory, n2 is too long to wait
 - Generates duplicate nodes in cyclic graphs

How many more nodes IDA* expand than A*

Memory limited heuristic search

- IDA* uses very little memory – linear memory
- Other algorithms may use more memory for more efficient search

Recursive BFS

- Uses any linear space and mimic best first search (better than IDA*)
- It keeps track of the f-value of the best alternative path available from any ancestor of the current node

MA*

- MA* and SMA* are restricted memory best first search algorithms that utilize all the memory available
- The algorithm executes best first search while memory is available
- When the memory is full the worst node is dropped but the value of the forgotten node is backed up at the parent

Local Search

- Local search methods work on complete state formulations. They keep only a small number of nodes in memory
- Local search is useful for solving optimization problem
 - Often it is easy to find a solution
 - But hard to find the best solution
- Algorithm goal:
 - Find optimal configuration (e.g. TSP)

Iterative Improvement

- In many optimization problem, path is irrelevant, the goal state itself is the solution
 - Find configuration satisfying constraints (e.g. n-queens)

Local sEARCH

- Iterative methods
 - Start with a solution
 - Improve it towards a good solution

Example

Hill climbing

- Or gradient descent/ascent
- Iteratively maximize “value” of current state, by replacing it by successor state that has highest value, as long as possible

Example

- Complete state formulation for 8 queens
- Successor function: move a single queen to another square in the same column
- Cost: number of pairs that are attacking each other
- Minimization problem

Hill climbing

- Problem: depending on initial state, may get stuck in local extremum

Minimizing energy

- Compare our state space to that of a physical system that is subject to natural interactions
- Compare our value function to the overall potential energy E of the system
- On every updating, we have

Local Minimum Problem

Consequences of occasional ascents

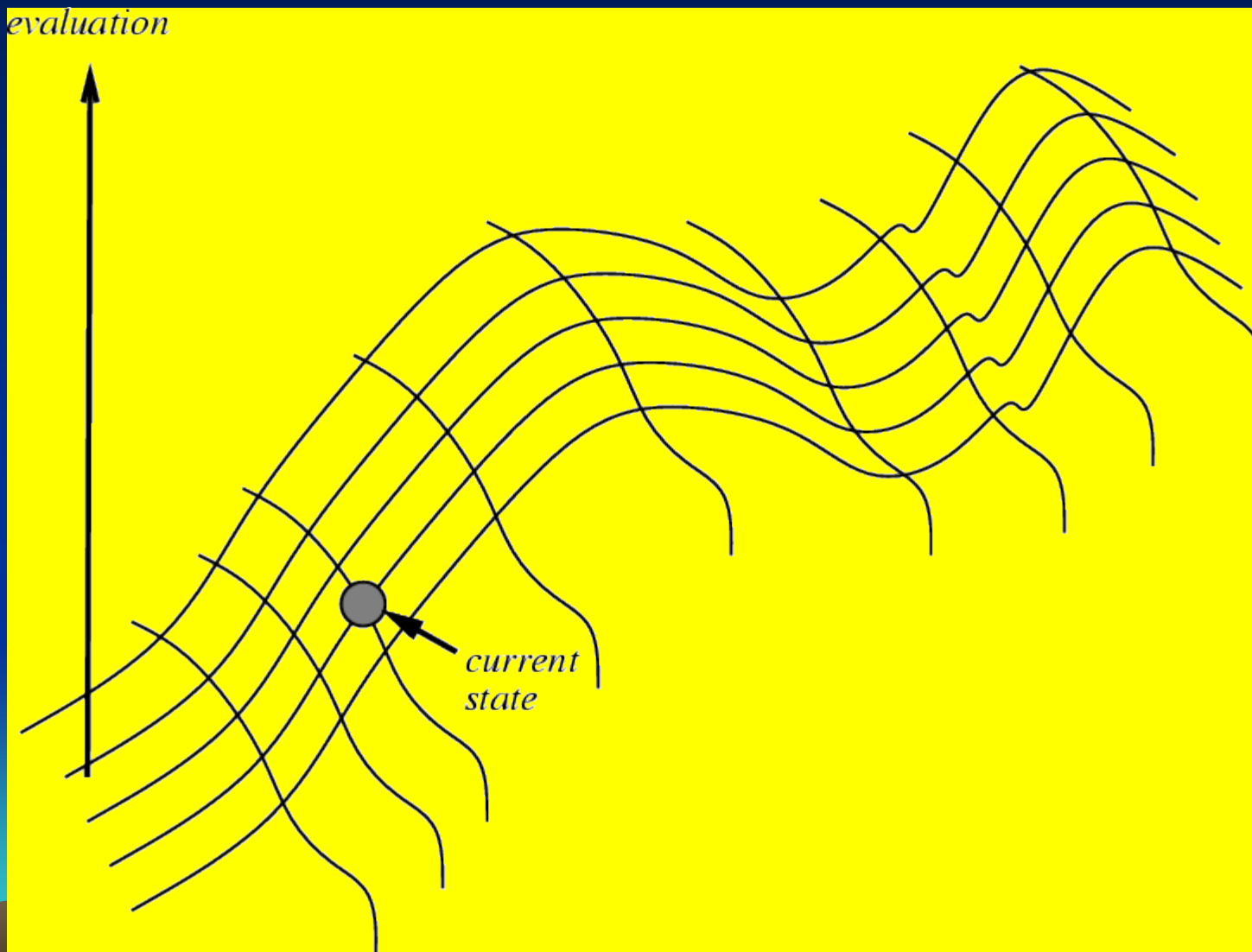
Simulated Annealing

- Basic idea: otherwise, do not give up, but instead flip a coin and accept the transition with a given probability (that is lower as the successor is worse)
- So we accept to sometimes “unoptimise” the value function a little with a non-zero probability

Iterative Improvement Search

- Another approach to search involves starting with an initial guess at a solution and gradually improving it until it is one.
- Some examples:
 - Hill Climbing – always try to make changes that improve the current state
 - Simulated Annealing – sometimes make changes that make things worse, at least temporarily
 - Genetic algorithm

Hill Climbing on a Surface of States

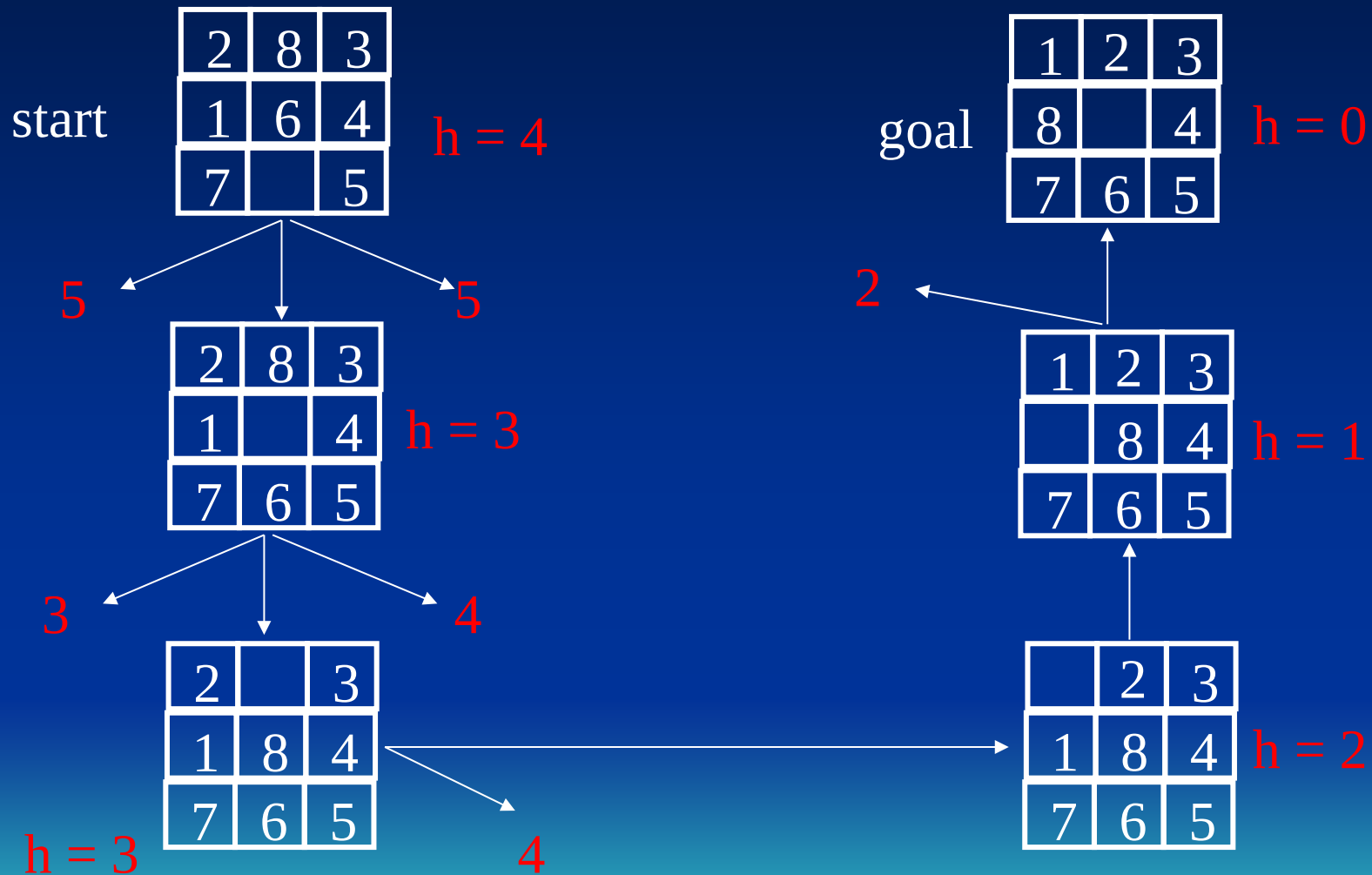


Height Defined
by
Evaluation
Function

Hill Climbing Search

- If there exists a successor n' for the current state n such that
 - $h(n') < h(n)$
 - $h(n') \leq h(t)$ for all the successors t of n ,
- then move from n to n' . Otherwise, halt at n .
- Looks one step ahead to determine if any successor is better than the current state; if there is, move to the best successor.
- Similar to greedy search in that it uses h only, but does not allow backtracking or jumping to an alternative path since it doesn't "remember" where it has been.
- $OPEN = \{\text{current-node}\}$
- Not complete since the search will terminate at "local minima," "plateaus," and "ridges."

Hill climbing example



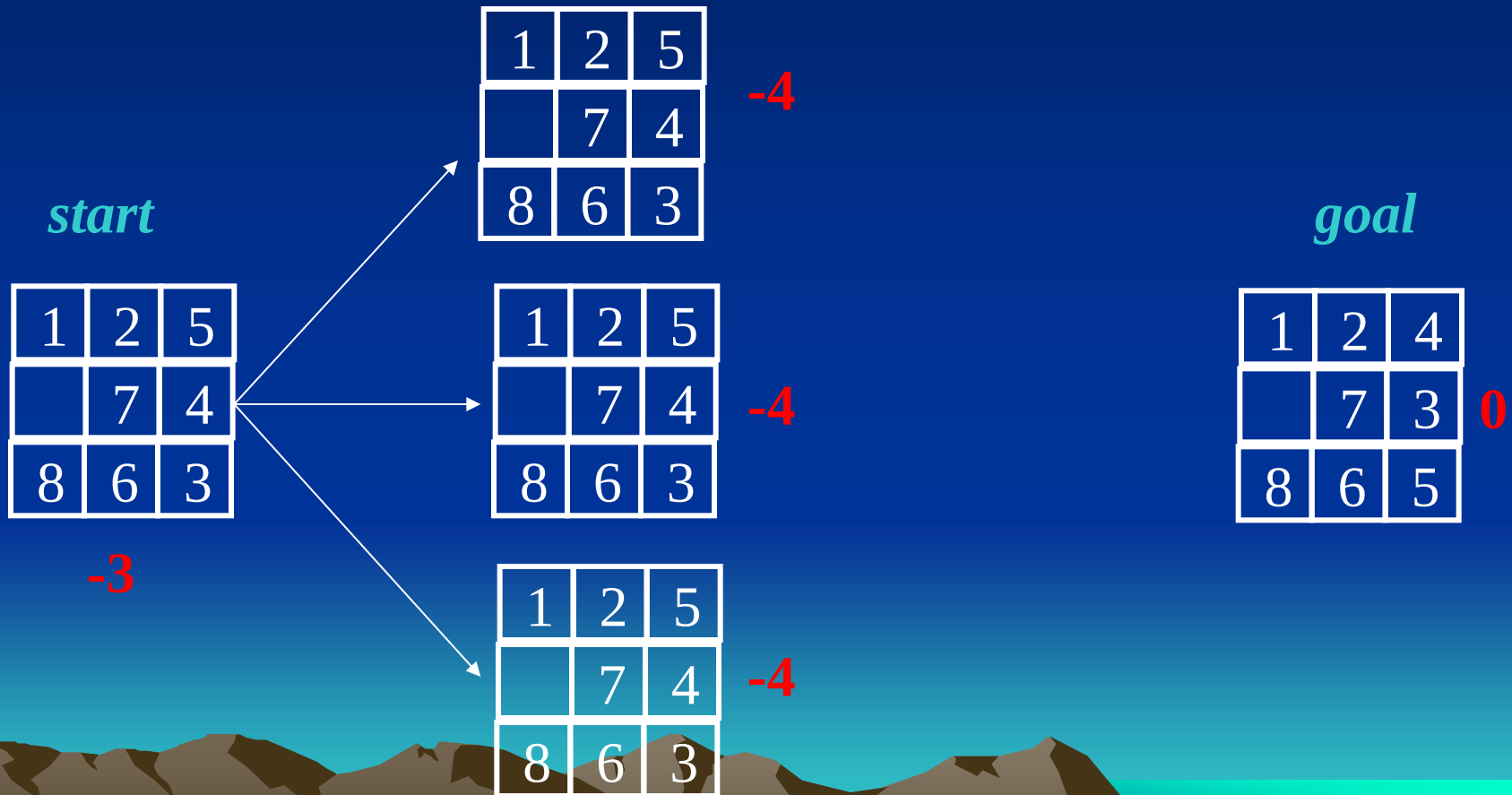
$f(n) = (\text{number of tiles out of place})$

copyright@Dr. M.S. Hossain

Drawbacks of Hill-Climbing

- Problems:
 - **Local Maximum** is a state that is better than all its neighbors but is not better than some other states farther away
 - **Plateau** is a flat area of the search space in which a whole set of neighbouring states have the same value.
 - **Ridges:** is a special kind of local maximum. It is an area of the search space that is higher than surrounding areas and that itself has a slope (which one would like to climb)..
- In each case, the algorithm reaches a point at which no progress is being made.
- Remedy:
 - Backtrack to some earlier node and try going in a different direction
 - Start again from a different start point
- Some problems spaces are great for Hill Climbing and others horrible.

Example of a local maximum



Example from Knight's Book

- Chapter 3, Section 3.2.2 (Page 68, 69)
- First heuristic
 - Add one point for every block that is resting on the thing it is supposed to be resting on. Subtract one point for every block that is sitting on the wrong thing (Local, why show?)
- Second heuristic
 - For each block that has the correct support structure (i.e. the complete structure underneath it is exactly as it should be) and on epoint for every block in the support structure. For each block that has an incorrect support structure, subtract one point for every block in the existing support structure.
- Give an example of how local maximum can be overcome by changing heuristic.

Simple Hill Climbing

1. Evaluate the initial state. If it is also goal state, then return it and quit. Otherwise Continue with the initial state as the current state.
2. Loop until a solution is found or until there are no new operations left to be applied in the current state:
 - (a) Select an operator that has not yet been applied to the current state and apply it to produce a new state
 - (b) Evaluate the new state:
 - i. If it is a goal state, then return it and quit.
 - ii. If it is not a goal state but it is better than the current state, then make it the current state
 - iii. If it is not better than the current state, then continue in the loop.

Steepest-Ascent Hill Climbing

- A useful variation on simple hill climbing considers all the moves from the current state and selects the best one as the next state

Compare HC (Steepest –ascent) and BFS

- In hill climbing, one move is selected and all the others are rejected, never to be reconsidered.
 - This produces the straight line behavior that is characteristics of hill climbing.
- In best-first search, one move is selected, but the others are kept around so that they can be revisited later if the selected path becomes less promising
- The best available state is selected in best-first search, even if that state has a value that is lower than the value of the state was just explored. This contrast with hill climbing, which will stop if there are no successor states will better values than the current state

Simulated Annealing

- Simulated Annealing (SA) exploits an analogy between the way in which a metal cools and freezes into a minimum energy crystalline structure (the annealing process) and the search for a minimum in a more general system.
- Each state n has an energy value $f(n)$, where f is an evaluation function
- SA can avoid becoming trapped at local minimum energy state by introducing **randomness** into search so that it not only accepts changes that decreases state energy, but also some that increase it.
- SAs use a control parameter T , which by analogy with the original application is known as the system “**temperature**” irrespective of the objective function involved.
- T starts out high and gradually (very slowly) decreases toward 0.

idea: escape local maxima by allowing some "bad" moves
but gradually decrease their size and frequency

function **SIMULATED-ANNEALING**(*problem*, *schedule*) returns a solution state

inputs: *problem*, a problem

schedule, a mapping from time to "temperature"

local variables: *current*, a node

next, a node

T, a "temperature" controlling prob. of downward steps

current ← MAKE-NODE(INITIAL-STATE[*problem*])

for *t* ← 1 to ∞ do

T ← *schedule*[*t*]

if *T* = 0 then return *current*

next ← a randomly selected successor of *current*

ΔE ← VALUE[*next*] - VALUE[*current*]

if $\Delta E > 0$ then *current* ← *next*

else *current* ← *next* only with probability $e^{\Delta E / T}$

Algorithm for Simulated Annealing

```
current := a randomly generated state;
T := T_0                                     /* initial temperature T0
    >>0 */
forever do
    if T <= T_end then return current;
    next := a randomly generated new state;    /* next !=
current */
     $\Delta E = f(\text{next}) - f(\text{current});$ 
    current := next with probability  $p_{\Delta E} = \frac{1}{1 + e^{\Delta E / T}};$ 

    T := schedule(T);                         /* reduce T by a cooling
    schedule */
```

Commonly used cooling schedule

- $T := T * \alpha$ where $0 < \alpha < 1$ is a cooling constant
- $T_k := T_0 / \log(1+k)$

Informed Search Summary

- **Best-first search** is general search where the minimum cost nodes (according to some measure) are expanded first.
- **Greedy search** uses minimal estimated cost $h(n)$ to the goal state as measure. This reduces the search time, but the algorithm is neither complete nor optimal.
- **A* search** combines uniform-cost search and greedy search: $f(n) = g(n) + h(n)$ and handles state repetitions and $h(n)$ never overestimates.
 - A* is complete, optimal and optimally efficient, but its space complexity is still bad.
 - The time complexity depends on the quality of the heuristic function.
 - IDA* reduces the memory requirements of A*.
- **Hill-climbing algorithms** keep only a single state in memory, but can get stuck on local optima.
- **Simulated annealing** escapes local optima, and is complete and optimal given a slow enough cooling schedule (in probability).
- **Genetic algorithms** escapes local optima, and is complete and optimal given a long enough evolution time and large population size.

Assignment

- Discuss and compare hill climbing and best-first search techniques
- Give an example of an admissible heuristic for the eight puzzle.
- Give three different heuristics for an $h^*(n)$ to be solved in solving the eight puzzle.
- Prove A^* is admissible. Hint: the proof should show that:
 - A^* search will terminate
 - During its execution there is always a node on Open that lies on an optimal path to the goal
 - If there is a path to goal, A^* will terminate by finding the optimal path
- Does admissibility imply monotonicity of a heuristic? If not, can you describe when admissibility would imply monotonicity?
- Prove that the set of states expanded by algorithm A^* is a subset of those examined by breadth-first search
- When would best-first search be worse than simple breadth-first search?